

CS415/515 Milestone 0 Report

BenchTime: A lab equipment reservation system

Luke McArthur & Jesse Seidel

lgmcarthur@crimson.ua.edu & jpseidel@crimson.ua.edu

Submitted September 2nd, 2026

Executive Summary

The Problem

Many of The University of Alabama's laboratory and maker spaces used for ECE classes such as the labs located in SERC use an informal system of reserving lab equipment and space. Often such a system is either non-existent or extremely informal i.e. making use of paper sign-up sheets or relying on class group chats. This often results in students traveling to a lab only to find out that the equipment they need is unavailable. If such a system does exist, it often lacks easy accessibility so last minute equipment reservation cancellations are impossible which results in equipment being left unused. If no system is being used, then lab staff have no record of how much each piece of equipment is being used and cannot make informed decisions about equipment maintenance schedules or future equipment purchases.

Proposed Solution

The solution to these problems is the creation of a web application(BenchTime) that provides authenticated role-based access to a shared equipment calendar. Lab managers would be able to create an account, create an equipment calendar, and invite students to this calendar. Students would also be able to create an account, view a list of calendars they have access to, and reserve equipment if it's not already reserved. BenchTime would prevent overlapping reservations automatically and would allow students to release a reservation last minute. Lab managers would have total control over their equipment calendars allowing them to remove certain reservations or students. BenchTime would also track and provide equipment utilization summaries.

Users

BenchTime intends to serve two primary groups of users. The first group is lab members/students who need to find and reserve lab equipment. The second group is lab managers who need to register equipment, resolve booking conflicts, and keep track of equipment use.

Target Users and Personas

Primary Users

Primary users are expected to be interacting with BenchTime regularly and as such the MVP is designed around their wants/needs. The primary users are:

1. Members: Students or possibly faculty/staff that are able to use equipment in a certain lab area. They can browse available equipment, reserve time, and manage their own reservations.
2. Lab Managers: Owners of the lab space. They can register/remove equipment, resolve reservation disputes, and review utilization statistics.

Personas

John Doe: a student in a 300 level ECE class. Primary User(Member).

John is working on a lab assignment for his ECE class that requires him to use a power supply. He must commute around forty-five minutes to his designated lab so it is imperative a power supply is available for him once he arrives.

Needs: John must know whether or not a power supply will be free before he begins his commute to the lab. Reserving a power supply must be easy and he needs to cancel his reservation if he cannot arrive on time.

Goals: To reserve a power supply for one hour after his estimated arrival time to the lab.

Constraints: John has unreliable transportation. If he cannot make it to the lab before his reservation, he will need to cancel it.

Jane Doe: Lab Owner. Primary User(Manager).

Jane manages ten machines inside of a lab commonly used by ECE students. She receives many complaints from people about the lack of a proper equipment reservation system and must deal with many scheduling conflicts.

Needs: An equipment reservation system that she can manage and provide to the members of her lab. This system must allow her to add the equipment she currently has in the lab and see the current list of reservations.

Goals: To reduce the number of scheduling conflicts she has to deal with.

Constraints: Jane does not have much of a software background. Therefore, managing the equipment calendar must be done through an easy-to-use web interface. She will not put up with having to create config files or access a database directly.

Proposed Application

Purpose

The primary purpose of BenchTime is to provide a centralized website where lab managers can create an equipment reservation calendar, add their lab equipment to it, and invite lab members to reserve equipment. Lab members can view a list of their current reservations and make future reservations if possible. The secondary purpose of BenchTime is to give lab managers access to equipment usage statistics to help them make maintenance and purchasing decisions. BenchTime intends to replace informal/non-existent equipment reservation methods with a user-friendly interface that can be accessed from anywhere.

Expected User Experience

BenchTime is designed primarily around short interactions. A member or lab manager using BenchTime should be able to quickly find what equipment is currently available in a given lab space, and make a reservation if needed with only a small number of clicks. BenchTime will use an account system which means that both members and lab managers must create a unique username with a password before accessing BenchTime's features. Members must be added to specific equipment calendars owned/created by lab managers before they can begin making reservations.

Major Features

For Members:

1. Account registration and account authentication upon sign-in.

2. View of available equipment calendars(ones the member was added to by a lab manager).
3. View of available equipment for reservation inside of a specific calendar.
4. View of available registration times for a specific piece of equipment.
5. Ability to reserve equipment if available.
6. Ability to remove an existing registration if needed.

For Lab Managers:

1. Account registration and account authentication upon sign-in.
2. View of owned equipment calendars.
3. Ability to add members using their username to an equipment calendar.
4. View of available equipment for reservation inside of a specific calendar.
5. Ability to add or remove pieces of equipment from a calendar.
6. View of available registration times for a specific piece of equipment.
7. Ability to remove any registration for a specific piece of equipment.
8. View of equipment usage statistics for a specific piece of equipment.

Overall System Features:

1. Account registration and authentication.
2. Storing account usernames/passwords and role(member or lab manager).
3. Prevention of overlapping registration times.
4. Storing created equipment calendars and their associated equipment.
5. Tracking and storing equipment usage information for specific calendars.

Intended Platform

BenchTime will be a responsive web application such that it does not require any installation nor a native client. It will be accessible from current versions of popular browsers such as Chrome, Safari, Firefox, and Microsoft Edge primarily from desktops/laptops.

Typical End-to-End Workflow

The following lists show a rough description of the typical workflow a member/lab manager can expect to experience.

Member:

1. They visit the BenchTime URL using a web browser.
2. They are taken to a login page that prompts them to enter their credentials(username/password) with an option to create an account.
3. If they do not have an account they click the option to create an account and are taken to a sign-up page.
4. At the sign-up page they will be prompted to sign up as a member or manager and to enter a username/password.
5. They select “member” and enter a username/password. If the username is not already taken they are redirected back to the login page.
6. They login using their newly created username/password.
7. After a successful login, they are taken to a page that shows their available lab equipment calendars(empty if they just created an account).
8. They wait for a lab manager to add them to a calendar using their username.

9. Once added they can click on the calendar which will redirect them to a page displaying a list of available equipment.
10. They select the equipment they would like to reserve.
11. The calendar displays the available reservation times for that piece of equipment.
12. They find a time that suits them.
13. They click a button to save the reservation.
14. They leave the website.

Lab Manager:

1. They visit the BenchTime URL using a web browser.
2. They are taken to a login page that prompts them to enter their credentials(username/password) with the option to create an account.
3. If they do not have an account they click the option to create an account and are taken to a sign-up page.
4. At the sign up page they will be prompted to sign up as a member or manager and to enter a username/password.
5. They select “manager” and enter a username/password. If the username is not already taken they are redirected back to the login page.
6. They login using their newly created username/password.
7. They are redirected to a list of the current calendars they own. If none are available an option exists for them to create one.
8. They can create a calendar and give it a name.
9. After creation they will be redirected to this new calendar.
10. They can add members using the member’s username to this calendar.

11. An option exists for them to add new equipment to this calendar.
12. They can see a list of available equipment.
13. After selecting a piece of equipment they can see the current reservations and the overall equipment usage statistics.
14. They can remove current reservations if needed.
15. They leave the website.

MVP Scope

Required MVP Features

Accounts and Access Control:

1. Member and lab manager account registration with username/password and authenticated sign-in.

Equipment:

1. Ability for members and lab managers to view a list of equipment associated with each equipment calendar.
2. Ability for lab managers to add/remove equipment from the calendars they own.

Reservations:

1. Ability for members to view, modify, and cancel only their own reservations.
2. Ability for members to create a new reservation for a specific piece of equipment.
3. Ability for lab managers to create a new equipment calendar.
4. Ability for lab managers to add members to an equipment calendar.

Platform:

1. Automated deployment of BenchTime on a local machine with the ability to access the above mentioned member/lab manager features.
2. Automated testing of some of BenchTime's features.
3. Deployment of BenchTime to a public URL with the above features and tested accessibility from a desktop/laptop.

Excluded MVP Features

1. Notifications. Notifications could be sent to members to remind them of an upcoming reservation. While this would be a useful feature it is somewhat out of the scope for an MVP.
2. Real-time server push. A periodic client refresh will be good enough to demonstrate the above features without needing a persistent connection.
3. Multiple lab managers on the same calendar. Useful if a lab is maintained by multiple people. However, this is not needed for minimal functionality.
4. Standardized web-page design/detailed pages. This is out of the scope for an MVP and will most likely be the focus of later milestones after basic functionality has been implemented.

Possible Features

1. University sign on. A way to ensure that lab managers can only see members from the university they are working for. This could be implemented by requiring a university email during account sign up.
2. Ability for lab managers to view equipment usage statistics and storing of usage statistics. While useful, it does not need to be included to achieve minimal functionality. Will be included in later milestones if it cannot be included in the MVP.
3. Recurring and repeating reservations. This would allow a member to set up many recurring/repeating reservations without having to do each manually.

4. Ability for lab managers to view, modify, and cancel reservations under the calendars they own. This is important but not necessarily for a minimum product. It should be included in later milestones, but may be included in the MVP if times allow.

Scope Boundaries

1. BenchTime exists purely as a reservation system and does not have any features to enforce proper use outside of the features given to the lab managers. As such, it is the duty of the lab managers to enforce correct usage.
2. Web only. BenchTime intends to be web only with no mobile or desktop client.
3. No financial transactions. BenchTime does not intend to include any features that would require financial transactions.
4. Minimal personal data. With the exception of possibly collecting a university name/email, BenchTime does not intend to store other personal information.
5. BenchTime requires that every hardware calendar have one lab manager and there will be no support for large organizations such as an entire university.
6. Though it makes use of equipment calendars, BenchTime does not intend to replace traditional calendar software like Outlook when it comes to event/meeting scheduling. It only intends to act as a reservation system.

Initial Requirements

Prioritized User Stories

The following requirements are prioritized using three levels:

- High: Must be included in the MVP.
- Medium: Improves the product a lot. Should be included in milestone 2.
- Low: Valuable and could be added if time allows for it. Can be in milestone 3.

Points on a 1-5 scale are calculated based on how complex features will be to implement, not how much time they will take.

Table 1: User Stories

ID	Story	Priority	Points
US-1	As a prospective member, I want to register as a member with a unique username and password so that I can begin reserving equipment.	High	5
US-2	As a member, I want to be added to an equipment calendar by a lab manager so I can begin reserving equipment.	High	3
US-3	As a member, I want to see all available machines in a given equipment calendar.	High	3
US-4	As a member, I want	High	4

	to reserve a piece of equipment during an open time slot so that it is reserved for me when I arrive.		
US-5	As a member, I want to cancel a reservation I no longer need so that someone else can reserve during that time.	Medium	2
US-6	As a prospective lab manager, I want to register as a lab manager with a unique username and password so that I can begin making equipment calendars.	High	5
US-7	As a lab manager, I want to create an equipment calendar so that I can begin inviting members to make reservations.	High	3
US-8	As a lab manager, I want to invite members to an equipment calendar so that they can begin making reservations.	High	3
US-9	As a lab manager, I want to be able to add/remove equipment from my equipment calendar so members can make reservations.	High	4
US-10	As a lab manager, I want to be able to edit	Medium	3

	the reservations under the equipment calendars I own so that I can resolve conflicts and misuse.		
US-11	As a manager, I want to view some utilization statistics for every piece of equipment so that I can make informed decisions about purchases and maintenance.	Medium	5
US-12	As a member, I would like to set up a recurring/repeating reservation so that I can avoid having to enter many reservations manually.	Low	3
US-13	As a lab manager, I would like to login using my university email so that I can only add members to equipment calendars that are from my university.	Low	5

Acceptance Criteria For High-Prority User Stories

US-1

- A prospective member must be able to navigate to a sign-up page where they can register as a member.
- This sign-up page must require the prospective member to enter a unique(not used by any other member or lab manager) username and password.

- This member's information must be stored in a database so that it can be retrieved later.

US-2

- A registered member must be able to be added to any equipment calendar by a lab manager.
- Once added to this equipment calendar, the member must be able to interact with features according to US-3 and US-4.

US-3

- A registered member that has been added to an equipment calendar must be able to see all equipment added to that calendar by the lab manager which is retrieved from a database.

US-4

- A registered member that has been added to an equipment calendar must be able to view the current registrations for specific pieces of equipment.
- The member must be able to create new reservations for a specific piece of equipment.
- This reservation, if it does not overlap with other reservations, should be stored in a database and linked to the current equipment calendar.

US-5

- A registered member that has been added to an equipment calendar must be able to remove their reservations under specific pieces of equipment.
- This removal will be removed from the database so it is present for other users.

US-6

- A prospective lab manager must be able to navigate to a sign-up page where they can register as a manager.
- This sign-up page must require the prospective lab manager to enter a unique(not used by any other member or lab manager) username and password.

- This lab manager's information must be stored in a database so that it can be retrieved later.

US-7

- A registered lab manager must be able to create a new equipment calendar which they can interact with according to US-8 and US-9.
- This equipment calendar must be stored in a database so that it may be retrieved later.

US-8

- A registered lab manager must be able to view an equipment calendar that they have created.
- The lab manager must be able to add members to their current equipment calendar based off of the member's username.
- The member's access to this calendar must be stored in a database so that it can be retrieved later.

US-9

- A registered lab manager must be able to view an equipment calendar that they have created.
- A lab manager must be able to add or remove reservable equipment from this calendar.
- The additional or removal of equipment must be stored in a database and linked to this specific equipment calendar.

Functional Requirements

Table 2: Functional Requirements

ID	Description
FR-1	The system shall allow member/lab manager

	registration with a unique username and password.
FR-2	The system shall authenticate members by username and password.
FR-3	The system shall support two roles: member and lab manager and shall enforce role permissions for every operation.
FR-4	The system shall restrict the modification(creation and cancelation) of a reservation to its member owner and the lab manager.
FR-5	The system shall maintain a list of equipment and link it to every equipment calendar .
FR-6	The system shall maintain a list of equipment calendars linked to their lab manager owner.
FR-7	The system shall allow managers to add and remove equipment from their equipment calendars.
FR-8	The system shall allow an authenticated member to reserve a time slot for a specific piece of equipment.
FR-9	The system shall allow a lab manager to add a member to an equipment calendar they own.

Non-Functional Requirements

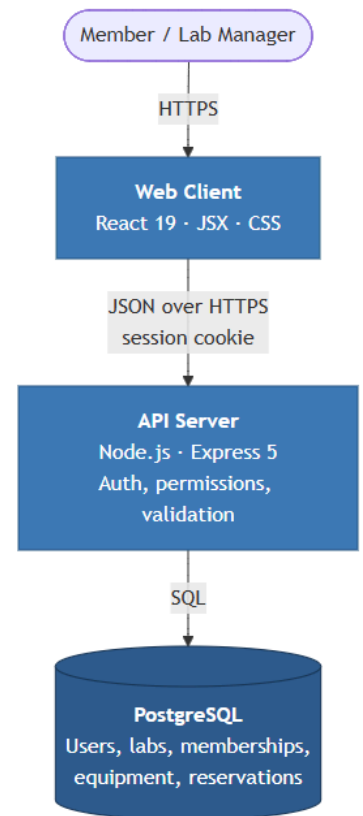
Table 3: Non-Functional Requirements

ID	Description
NFR-1	The system shall only collect a username, password, and possibly an email(see US-13). No other personal or identifiable information shall be collected.
NFR-2	The system shall store passwords using a hashing algorithm NOT using plaintext.
NFR-3	No secret or credential shall be committed to version control. Any configuration file must be supplied via a .env file with documentation.

NFR-4	All website functionality shall be achievable via a desktop/laptop with a keyboard and mouse alone.
NFR-5	A newly registered member shall be able to complete a reservation for a specific piece of equipment in no more than five interactions beginning from the sign-in page.
NFR-6	Any rejected action from a member or lab manager shall result in an error message stating the reason for the rejection.

Initial Design and Architecture

BenchTime will use a three-tier client-server architecture. We will use React 19, JSX, and CSS for the client[1][2][3]. We will use Node.js and Express 5 for the backend, and the backend will be the only part of the system that touches the database[5][6]. We picked React and Node partly because we are already using both for the other project in this class, so what we learn there carries over instead of splitting our time between two unfamiliar stacks. We chose PostgreSQL because our data is relational and because it can reject two overlapping reservations at the database level instead of making the backend check first[7]. We considered SQLite since it is simpler to set up, but it cannot enforce that overlap rule, which would leave us handling the conflict in application code where two requests can both pass the check before either one saves[8]. All permission checks will happen on the server, so hiding buttons on the client is only for convenience and not for security. Figure 1. High-level/initial architecture diagram(see right).



Data and External Resources

Our main data entities(see Table 4) are User, Lab, Membership, Equipment, and Reservation. A User has a username, a password, and a role of either member or manager. A Lab is an equipment calendar owned by one manager, and a Membership is the record that gives a member access to a lab. Equipment belongs to a lab, and a reservation belongs to one piece of equipment and one user and stores the start and end time of the booking. Removing equipment will be a soft delete so old reservations stay intact for the usage statistics we plan to add after the MVP.

BenchTime does not need any outside datasets, APIs, or hardware. All of the data in the system is created by our own users, which removes a lot of risk since we are not depending on anyone else's service staying available. The libraries and tools we plan to use are free and open source, so the project should not cost us anything. One thing we would like to look into later is using the university's single sign-on so members could log in with their school account instead of a separate username and password. That would tie accounts to real students and make it easier for a manager to know who they are adding, but it depends on whether we can get access to it, so it is not part of the MVP. For now we are only storing a username, a password, a role, and reservation times.

Table 4: Entities

Entity	Attributes	Relationships
User	Id, username, password, role	Many Labs (Owner), Many Memberships, Many Reservations
Lab	Id, name, owner	One User (Owner), Many Memberships, Many Equipment
Membership	Id, lab, user	One User, One Lab
Equipment	Id, name, description, lab, active	One Lab, Many Reservations
Reservation	Id, equipment, user, startTime, endTime	One equipment, One User

Agile Development Process

We will work in two week sprints, which gives us about six sprints across the semester and two sprints leading into each milestone. Two weeks is short enough that a bad decision only costs us one sprint, but long enough to finish a feature that touches the database, the backend, and the client. At the start of each sprint we will pick the work for that sprint, and at the end we will demo what we built to each other and talk about what to change next time. Since there are only two of us, we will keep the process light and check in over text a few times a week instead of holding daily standups.

We will keep a prioritized backlog and track every piece of work in an issue tracker rather than working off a shared document or from memory. We plan to start with GitHub Issues and a project board since our repository is already on GitHub, but we are not locked into that specific tool and may switch to something else if it fits us better once we get going. What matters to us is that the process stays the same either way. Every piece of work gets written down before anyone starts on it, each item is written as a user story with acceptance criteria, so we agree on what finished means, and items are ordered by priority and grouped by which milestone they belong to.

For version control we will protect the main branch so nothing can be pushed to it directly. All work happens on a short branch named after its issue, and gets merged through a pull request that the other person reviews and approves. Branches should not live more than a few days so reviews stay small. When reviewing, we will check that the feature actually does what the issue asked for, that permission checks are on the server, that user input is validated, and that there are tests for the new code. We will respond to each other's pull requests within a day so review never becomes the thing holding us up, and we will tag main at each milestone.

Our Definition of Done is that a task is not finished until:

1. It does what the issue asked and we have tested it in a running version.
2. It has been reviewed and merged into main through a pull request.
3. It has tests, and the whole test suite still passes.
4. The app still runs from a fresh clone using the setup instructions in the README.

That last one is what keeps us honest about the requirement that every milestone has a working version, since main is always runnable.

Milestone Plan

We plan to work in two week sprints, with two sprints leading into each milestone. Table 5 shows what we expect to deliver at each one.

Table 5: Milestone plan

Milestone	Due	Sprints	Delivarables
1	9/30	Sep 2-15, Sep 16-29	A working MVP where a manager can create a lab, add equipment, and add members, and a member can sign in, view equipment, and create and cancel their own reservations. Overlapping reservations are rejected. Includes our updated requirements, use cases, design and architecture documents, wireframes, an architecture decision record, and setup instructions the TA can follow.
2	11/2	Sep 30-Oct 13, Oct 14, Nov 1	Everything from Milestone 1 still working, plus equipment usage statistics for managers. Includes unit and integration tests with a coverage report, evidence of peer code review, a changelog, and two tagged releases
3	11/30	Nov 2-15, Nov 16-29	The finished application deployed to a public URL. A CI pipeline that blocks merges when tests fail, a CD pipeline that deploys on merge, a metrics report, a polished README, and a live demo.

Team Responsibilities

Instead of splitting the work into frontend and backend, we are splitting it by feature so that each of us builds a feature all the way through, from the database to the API to the screen the user sees. We chose this because it means neither of us is blocked waiting on the other to finish their half, and both of us end up understanding the whole stack instead of only one layer of it.

Table 6 shows how we are dividing things up to start.

Table 6: Responsibilities

Feature	Person
Accounts, sign in, permissions	Luke
Labs, adding members to labs	Jesse
Deployment, CI/CD	Luke
Equipment management	Jesse
Reservations with calendar view	Luke
Usage stats for managers	Jesse
Schema, Backlog, Code Review, README	Both

We will set up the project and design the database schema together at the start, since almost every feature depends on it and it is easier to agree on it once than to fix it later. After that, each person leads their own features and the other person reviews the pull requests. The downside of splitting by feature is that we will both be touching the same parts of the codebase, so we are keeping branches small and short lived and telling each other before we change something shared. For work neither of us has done before, mainly the deployment pipeline, we will do the first pass together so we both understand it rather than one person being the only one who can fix it. Whoever builds a feature also writes its tests and documentation.

Risks and Mitigation

Table 7: lists the main risks we expect and what we plan to do about them.

Risk	Type	Impact	How to Mitigate
People booking the same equipment at the same time.	Technical	High	PostgreSQL will reject overlap.
Midterms and fall break close to milestone deadlines.	Schedule	Medium	Plan ahead and make sure to do work early so we aren't crammed at the end.
A user can access a lab they aren't a part of.	Security	High	Ensure every endpoint checks permission on the server side.
A bad migration corrupts our database.	Data	High	Test the migrations locally first.

Most of these come down to doing the risky work early instead of at the deadline. We will go back through this table at the end of each sprint and update it.

Success Criteria

BenchTime succeeds if at the end of the semester a person is able to access a public URL for the website, sign-up/sign-in, be added to an equipment calendar by a manager, and reserve a piece of equipment. The BenchTime repository should be able to be cloned and reproduce this same behavior while running on a local machine. We define three different levels of success:

Minimum: All “high priority” user stories are complete and demonstrable via the public URL. All functional and non-functional requirements with these user stories are completed and demonstrable. Here “complete” is defined as meeting our definition of done listed under “Agile Development Process”.

Target: The requirements for minimum success have been achieved plus all “medium priority” user stories and their functional/non-functional requirements are complete and demonstrable. Here “complete” is defined as meeting our definition of done listed under “Agile Development Process”.

Stretch: Target success plus at least one user story labeled “low priority” and their functional/non-functional requirements are complete and demonstrable. Here “complete” is defined as meeting our definition of done listed under “Agile Development Process”.

References

1. *React V19*. React. (n.d.). <https://react.dev/blog/2024/12/05/react-19>
2. *Introducing JSX*. React. (n.d.-a). <https://legacy.reactjs.org/docs/introducing-jsx.html>
3. MozDevNet. (n.d.). *Web*. MDN Blog RSS.
<https://developer.mozilla.org/en-US/docs/Web/CSS>
4. *Node.js - run javascript everywhere*. Node.js -. (n.d.). <https://nodejs.org/en>
5. *Express.js · node.js web application framework*. Express.js. (n.d.). <https://expressjs.com/>
6. Group, P. G. D. (2026, September 3). PostgreSQL. <https://www.postgresql.org/>
7. Sqlite Home Page. (n.d.). <https://sqlite.org/index.html>

Other Deliverables

Github Link

<https://github.com/luke-mca/BenchTime>

Initial README.md

<https://github.com/luke-mca/BenchTime/blob/main/README.md>

Prioritized BACKLOG.md

<https://github.com/luke-mca/BenchTime/blob/main/BACKLOG.md>

Contribution Statements

Luke McArthur — Wrote the initial design and architecture, data model, agile development process, milestone plan, team responsibilities, and risk analysis, and set up the repository, README, backlog, and architecture diagram.

Jesse Seidel — Wrote the executive summary, target users and personas, proposed application, MVP scope, initial requirements (user stories, acceptance criteria, and the functional and non-functional requirement tables), and the success criteria.